

❏ Binary Heap Notes

A **binary heap** is a **complete binary tree** used to efficiently implement **priority queues**.

◆ Types

- **Max Heap:** Parent $\geq$ children (root = largest element)
 - **Min Heap:** Parent $\leq$ children (root = smallest element)
-

◆ Properties

- **Shape Property:** Must be a *complete binary tree* (all levels filled except possibly the last). left to right.
 - **Heap Property:** For a max heap, every parent node is greater than or equal to its children.
-

◆ Common Operations

Operation	Description	Time Complexity
<code>insert(x)</code>	Add element and bubble up	$O(\log n)$
<code>getTop()</code>	Return min/max element	$O(1)$
<code>extractTop()</code>	Remove top and heapify down	$O(\log n)$
<code>heapify()</code>	Build heap from an array	$O(n)$
<code>isEmpty()</code>	Check if heap is empty	$O(1)$

◆ Implementation Details

- Typically implemented using an **array**.
 - Parent index: i
 - Left child: $2*i + 1$
 - Right child: $2*i + 2$
 - Parent: $(i - 1) / 2$
-

◆ Space Complexity

- $O(n)$ — stores all elements in an array.
-

◆ Applications

- Priority Queues
 - Heap Sort
 - Graph Algorithms (Dijkstra's, Prim's)
 - Order Statistics (k-th smallest/largest element)
-

◆ Heap Sort Summary

- **Steps:** Build heap → repeatedly extract top
 - **Time:** $O(n \log n)$
 - **Space:** $O(1)$ (in-place for arrays)
 - **Stability:** Not stable
-

◆ Why There Is $O(n)$ and $O(n \log n)$ Heap Creation

There are two ways to create a heap:

1. Bottom-up Heapify ($O(n)$):

- Used when building a heap from an existing array.
- The algorithm calls `heapify()` on all non-leaf nodes, starting from the last parent down to the root.
- Most nodes are near the bottom and require little adjustment, resulting in **linear time $O(n)$** overall.

2. Repeated Insertion ($O(n \log n)$):

- Insert each element one by one into an initially empty heap using `insert()`.
- Each insertion takes $O(\log n)$ (bubble up).
- For n elements, total = **$O(n \log n)$** .

✓ In short:

Bottom-up heapify builds faster ($O(n)$), while inserting elements one-by-one is slower ($O(n \log n)$). ``